

# Manual 10 - Unión Completa (Todos los Manuales en Uno)

---

**Versión:** 1.0

**Fecha:** 2026-05-14

**Descripción:** Este documento es la unión de todos los manuales (00 al 09) sin cambios en sus contenidos originales. Contiene desde índice hasta guías operacionales, técnicas y de usuario en un solo archivo.

---

## MANUAL 00 - Manuales de Implementacion - RoomOps Front

---

Version: 1.0 Fecha: 2026-05-14 Publico: tecnico y no tecnico

### Objetivo de esta carpeta

Esta carpeta reúne manuales separados para entender como se implemento el sistema de vistas, autenticacion, sesion, permisos por rol y acciones permitidas. Cada documento incluye explicacion no tecnica, explicacion tecnica y ejemplos practicos.

Cada manual tiene dos enfoques:

- Explicacion no tecnica: para operacion, jefaturas o personas de negocio.
- Explicacion tecnica: para desarrollo, QA y soporte.

### Mapa de manuales

#### 1. 01\_VISTAS\_Y\_NAVEGACION.md

- Rutas, estructura de pantalla, menu lateral y visibilidad de vistas.

#### 2. 02\_AUTENTICACION\_Y\_SESION.md

- Login, token JWT, sesion local, expiracion y cierre de sesion.

#### 3. 03\_ROLES\_Y\_PERMISOS\_RBAC.md

- Matriz de permisos por rol y restricciones de acciones.

#### 4. 04\_DASHBOARD\_TRABAJADOR.md

- Dashboard personal del trabajador y logica de datos mostrados.

#### 5. 05\_GUARDADO\_DE\_TAREAS\_Y\_CHECKLIST.md

- Como se guardan cambios desde panel de detalle y fallback local.

#### 6. 06\_TIPOS\_DE\_TAREA\_Y\_REGLAS.md

- Reglas por tipo de tarea (Aseo, Mantencion, Repaso).

## 7. 07\_PRUEBAS\_Y\_VALIDACION.md

- Casos de prueba funcionales y tecnicos para validar todo el flujo.

## 8. 08\_MANUAL\_DE\_USUARIO.md

- Manual de uso diario con anexos separados por rol: ADMIN, SUPERVISOR y TRABAJADOR.

## 9. 09\_IMPLEMENTACION\_COMPLETA.md

- Documento tecnico integral con detalles end-to-end y ejemplos de codigo implementado: rutas, sesiones, permisos, graficos, tablas, notificaciones, guardado y fallback.

## 10. 10\_UNION\_COMPLETA.md (Este archivo)

- Union de todos los manuales (00 al 09) sin cambios en sus contenidos.

## Orden recomendado de lectura

1. Primero 01 y 02 para entender flujo base de la aplicacion.
2. Luego 03 para comprender permisos y limites por rol.
3. Despues 04, 05 y 06 para operacion diaria y reglas de negocio.
4. Finalmente 07 para pruebas, QA y soporte.

## Nota importante

Estos manuales describen el estado actual de implementacion del frontend. Si se modifican permisos, rutas o contratos de API, se recomienda actualizar estos documentos en la misma tarea de desarrollo.

---

# MANUAL 01 - Vistas y Navegacion

---

Version: 1.1

## 1) Explicacion no tecnica

Que problema resuelve

Evita dos problemas operativos comunes:

- Que un usuario vea pantallas que no le sirven (ruido y confusion).
- Que un usuario sin permiso intente hacer acciones que no debe.

Como funciona en simple

La aplicacion tiene un "portero" de entrada y un "menu inteligente":

- Portero: si no hay sesion valida, te manda a Login.
- Menu inteligente: muestra solo modulos permitidos por rol.

## Ejemplo no tecnico 1

Escenario:

- Ana (TRABAJADOR) inicia sesion.

Resultado esperado:

- Ve Inicio personal.
- Ve Tareas.
- Ve Kanban personal.
- No ve Usuarios ni Apartamentos.

## Ejemplo no tecnico 2

Escenario:

- Carlos (SUPERVISOR) inicia sesion.

Resultado esperado:

- Ve Dashboard general.
- Ve Usuarios (lectura), Tareas, Kanban y Apartamentos.

## 2) Explicacion tecnica

Archivos clave

- src/App.jsx
- src/components/sidebar/Sidebar.jsx
- src/service/permissions.js

Como se protege la navegacion

En src/App.jsx se usan dos wrappers:

1. RequireAuth
  - Bloquea rutas privadas cuando no hay sesion valida.
2. RequireGuest
  - Evita que un usuario ya autenticado vuelva a /login.

Rutas activas

- /login
- /
- /home
- /tasks
- /users
- /apartments

- /kanban

## Layout compartido

El Layout en src/App.jsx renderiza:

- Sidebar
- Header mobile
- Outlet con la vista actual

Con esto, todas las vistas internas comparten el mismo marco de navegacion.

## Logica del menu

En src/components/sidebar/Sidebar.jsx se evalua:

- canViewUsers()
- canViewApartments()
- canViewAnyTasks()
- canViewAnyKanban()
- canViewAdminDashboard()

Con eso se define que items mostrar y como se etiqueta el primer item:

- Dashboard (admin/supervisor)
- Inicio (trabajador)

## 3) Flujo completo (paso a paso)

1. Usuario abre la app.
2. App valida token/sesion.
3. Si no es valido, redirige a /login.
4. Si es valido, entra al Layout.
5. Sidebar consulta permisos y dibuja menu.
6. Usuario navega a modulo permitido.
7. La vista aplica filtros por permisos sobre los datos.

## 4) Ejemplo tecnico guiado

Caso: usuario no autenticado entra a /tasks

Entrada:

- URL: /tasks
- Estado: sin token valido

Salida:

- RequireAuth devuelve Navigate a /login.

Caso: trabajador autenticado

Entrada:

- Rol detectado: TRABAJADOR

Salida en Sidebar:

- Muestra Inicio, Tareas, Kanban.
- Oculta Usuarios y Apartamentos.

## 5) Errores comunes y como detectarlos

- Error: aparece menu vacio
  - Revisar getUserRole() y currentUser en localStorage.
- Error: usuario entra a vista no esperada
  - Revisar helper de permisos usado en Sidebar.
- Error: ruta publica accidental
  - Verificar que la ruta este dentro del bloque RequireAuth.

## 6) Checklist para agregar una vista nueva

1. Crear ruta en src/App.jsx.
2. Definir permiso en src/service/permissions.js.
3. Agregar helper semantico (ejemplo: canViewReports).
4. Mostrar item en Sidebar solo con ese helper.
5. Aplicar filtro de datos por permisos dentro de la vista.
6. Agregar caso de prueba por rol.

---

# MANUAL 02 - Autenticacion y Sesion

---

Version: 1.1

## 1) Explicacion no tecnica

Que hace esta parte

Resuelve 3 preguntas clave:

- Quien puede entrar.
- Cuanto tiempo permanece adentro.
- Que pasa cuando el acceso deja de ser valido.

Flujo simple para cualquier persona

1. El usuario escribe correo y clave.
2. El servidor responde si las credenciales son validas.

3. Si son validas, la app guarda una "llave temporal" (token).
4. Con esa llave se habilita la navegacion interna.
5. Cuando la llave vence, la app cierra sesion automaticamente.

## Ejemplo no tecnico

Escenario:

- Sofia inicia sesion a las 09:00.
- A las 12:00 su token ya vencio.
- Al abrir una vista, la app detecta token vencido y la envia a Login.

Resultado:

- No queda sesion "fantasma" abierta.
- Se evita que alguien use una sesion vieja.

## 2) Explicacion tecnica

Archivos clave

- src/components/login/Login.jsx
- src/service/localStorage.js
- src/service/api.js
- src/App.jsx

Flujo tecnico de Login

1. Login.jsx hace POST a /api/v1/auth/login.
2. Espera respuesta con formato: { token, user }.
3. setAuthToken(token) configura Authorization para llamadas futuras.
4. setUserSession(user) normaliza campos y guarda currentUser.
5. Se dispara evento authChanged para refrescar componentes.
6. navigate() redirige a la ruta de destino.

Ejemplo tecnico de request/response

Request:

```
{
  "email": "trabajador@roomops.com",
  "password": "123456"
}
```

Response esperada:

```
{
  "token": "eyJhbGciOiJI...",
}
```

```
"user": {  
  "id": 17,  
  "email": "trabajador@roomops.com",  
  "firstName": "Sofia",  
  "lastName": "Diaz",  
  "role": "TRABAJADOR",  
  "activo": true  
}
```

## Donde se guarda la sesion

En localStorage:

- token
- isLoggedIn
- currentUser

currentUser queda normalizado para evitar diferencias entre frontend y backend.

## Validacion de expiracion

isTokenExpired(token):

- decodifica payload JWT
- lee claim exp
- compara con Date.now()

isUserLoggedIn():

- valida flag de sesion
- valida token
- si no cumple, ejecuta clearUserSession()

## Fallback de robustez

Si user llega incompleto en Login.jsx:

- se decodifican claims del token
- se arma sesion minima con id/email/role

Esto evita que un 403 posterior en otro endpoint rompa la sesion inicial.

## 3) Flujo completo con ejemplos

### Caso A - Login exitoso normal

1. Usuario envia credenciales validas.
2. API devuelve token + user.
3. App guarda sesion.
4. Sidebar muestra modulos por rol.

## Caso B - Token vencido

1. Usuario mantiene pestaña abierta.
2. Token vence por tiempo.
3. En siguiente chequeo, `isUserLoggedIn` devuelve false.
4. App limpia sesion y redirige a `/login`.

## Caso C - Role con prefijo

Si backend envia `ROLE_SUPERVISOR`:

- `getUserRole` normaliza a `SUPERVISOR`.

Resultado:

- No falla la matriz de permisos por formato de string.

## 4) Problemas frecuentes y solucion

- Problema: usuario entra y no ve modulos
  - Revisar campo `role` en `currentUser`.
  - Revisar normalizacion en `getUserRole`.
- Problema: cierre de sesion inesperado
  - Verificar `exp` del token.
  - Verificar hora del sistema cliente.
- Problema: backend retorna 403 en consulta de usuario
  - Mantener sesion con datos de login.
  - Evitar depender de endpoint no permitido para completar sesion.

## 5) Buenas practicas de mantenimiento

1. Mantener contrato de login estable: `{ token, user }`.
2. Normalizar rol siempre en un solo lugar.
3. No duplicar logica de expiracion en muchos componentes.
4. Al cambiar auth backend, actualizar este manual y tests.

---

# MANUAL 03 - Roles y Permisos (RBAC)

---

Version: 1.1

## 1) Explicacion no tecnica

Que es RBAC en este proyecto

RBAC significa que cada rol entra a una "version controlada" de la app:



- ve solo lo que necesita
- hace solo lo que tiene permitido

## Ejemplo no tecnico

Escenario:

- Un trabajador quiere eliminar un usuario.

Resultado esperado:

- No ve la opcion y no puede ejecutar esa accion.

Escenario:

- Un supervisor abre Usuarios.

Resultado esperado:

- Puede revisar informacion, pero no tiene acciones administrativas completas.

## 2) Explicacion tecnica

Fuente unica de verdad

Archivo: src/service/permissions.js

Este archivo define:

- ROLES
- PERMISSIONS
- rolePermissions (matriz rol -> permisos)
- helpers de alto nivel (canViewUsers, canViewAnyTasks, etc)

Matriz resumida por rol

ADMIN / ADMINISTRADOR:

- Gestion total de usuarios, apartamentos, tareas, dashboard y kanban.

SUPERVISOR:

- Lectura de usuarios.
- Gestion operativa de tareas y checklist.
- Acceso a dashboard y kanban.

TRABAJADOR:

- Tareas propias.
- Cambio de estado/checklist en tareas asignadas.
- Kanban personal.

Helpers clave

- hasPermission(permission)
- canViewAllTasks()
- canViewOwnTasks()
- canEditSpecificTask(task)
- filterTasksByPermissions(tasks)
- canViewAnyTasks()
- canViewAnyKanban()

### 3) Ejemplos tecnicos concretos

#### Ejemplo A - Filtro de tareas

Entrada:

- Rol actual: TRABAJADOR
- tasks: 12 tareas mezcladas

Proceso:

- filterTasksByPermissions(tasks)
- internamente usa isTaskOwner(task)

Salida esperada:

- solo tareas asignadas al usuario autenticado

#### Ejemplo B - Propietario por distintos formatos

isTaskOwner(task) soporta:

- task.assignedUsers[]
- task.assignedUserId
- task.usuarioAsignadoId
- task.assignedUser.email

Esto cubre variaciones de contratos backend.

#### Ejemplo C - Menu lateral

Sidebar consulta canViewUsers().

Si false:

- no renderiza link de Usuarios.

Si true:

- renderiza link de Usuarios.

### 4) Donde impacta RBAC

- Navegacion: que links existen.

- Datos: que registros se ven.
- Acciones: que botones/operaciones aparecen.
- Dashboard: vista admin vs vista trabajador.

## 5) Diferencia entre control UI y seguridad real

Frontend RBAC:

- evita confusion y acciones no deseadas desde la interfaz.

Backend seguridad:

- impide tecnicamente operaciones no autorizadas (aunque alguien intente llamar API directo).

Ambas capas deben existir.

## 6) Checklist para cambiar permisos

1. Modificar rolePermissions en permissions.js.
2. Revisar helpers dependientes.
3. Probar Sidebar con cada rol.
4. Probar Task/Kanban/Home con cada rol.
5. Verificar reglas equivalentes en backend.

---

# MANUAL 04 - Dashboard del Trabajador

---

Version: 1.1

## 1) Explicacion no tecnica

### Objetivo

Mostrar al trabajador solo informacion accionable para su jornada.

### Que incluye

- Resumen rapido de carga diaria.
- Tarea mas urgente.
- Progreso del dia.
- Lista de tareas recientes.
- Accesos directos a modulos operativos.

### Ejemplo no tecnico

Escenario de inicio de turno:

- Marta abre la app a las 08:00.
- Ve 3 pendientes, 1 en progreso, 0 hechas hoy.
- El panel le muestra como urgente una tarea Aseo con fecha de hoy.

Resultado:

- Marta no necesita revisar listas largas.
- Sabe de inmediato por cual tarea comenzar.

## 2) Explicacion tecnica

Archivo principal

src/components/home/Home.jsx

Decision de vista

Home evalua canViewAdminDashboard():

- true -> dashboard administrativo
- false -> WorkerDashboard

Dataset base de WorkerDashboard

Se cargan en paralelo:

- getTasks()
- getApartments()
- getStatuses()

Luego:

- myTasks = filterTasksByPermissions(tasks)

Con esto, el trabajador nunca consume una lista global en pantalla.

## 3) Metricas y como se calculan

Pendientes

Cuenta tareas cuyo estado normalizado es pending.

En progreso

Cuenta tareas cuyo estado normalizado es in-progress.

Hechas hoy

Cuenta tareas en done cuya fecha coincide con el dia actual.

Progreso diario

Formula:

```
progressPercent = round((completedCount / totalMyTasks) * 100)
```

## Tarea urgente

Se busca entre pending + in-progress y se ordena por:

- fecha
- hora limite

La primera del orden se muestra como "Proxima tarea urgente".

## 4) Ejemplo tecnico con datos

Entrada filtrada del trabajador:

```
[
  {"id": 1, "titulo": "Aseo apto 301", "fecha": "2026-05-14", "dueTime": "16:00", "statusId": 1},
  {"id": 2, "titulo": "Repaso apto 204", "fecha": "2026-05-14", "dueTime": "16:00", "statusId": 2},
  {"id": 3, "titulo": "Mantencion apto 101", "fecha": "2026-05-13", "dueTime": "15:00", "statusId": 3}
]
```

Salida visual esperada:

- Pendientes: 1
- En progreso: 1
- Hechas hoy: 0
- Progreso: 33%
- Urgente: Aseo apto 301

## 5) Interaccion de detalle

Al hacer click en tarea:

- se abre TaskDetailPanel
- se muestran apartamento, asignado, fecha, hora limite, checklist

Los mapas apartmentNameById y statusNameById se pasan ya convertidos a texto para evitar errores de render.

## 6) UX aplicada

- Loading state explicito: "Cargando tus tareas..."
- Badges de tipo/prioridad/estado
- Indicador ATRASADA cuando vence fecha+hora
- Accion clara: boton "Ver detalle"

## 7) Mantenimiento y evolucion

Si agregas una metrica nueva:

1. Crear calculo con useMemo.
2. Agregar tarjeta visual.
3. Validar impacto en mobile.
4. Probar con tareas vacias y con alto volumen.

---

## MANUAL 05 - Guardado de Tareas y Checklist

---

Version: 1.1

### 1) Explicacion no tecnica

#### Problema original

El trabajador podia marcar checklist en el panel de detalle, pero al pulsar Guardar no se persistia.

#### Solucion implementada

Se conecto el panel de detalle con un callback real de guardado.

#### Comportamiento final:

- Si el servidor esta disponible, guarda en backend.
- Si el servidor falla, guarda en copia local para no perder trabajo.
- Al finalizar, recarga datos para reflejar estado real.

#### Ejemplo no tecnico

##### Escenario:

- Usuario marca 2 items como HECHO y 1 como BLOQUEADO con nota.
- Presiona Guardar.

##### Resultado esperado:

- Ve mensaje de exito.
- Al reabrir la tarea, los cambios siguen presentes.

### 2) Explicacion tecnica

#### Archivos involucrados

- src/components/home/Home.jsx
- src/components/taskDetail/TaskDetailPanel.jsx
- src/components/task/TaskFunctions.jsx
- src/service/taskService.js

#### Contrato entre panel y dashboard

TaskDetailPanel llama:

- onSaveChecklist(task, sanitizedChecklist)

Si no existe esa prop, el panel solo se cierra. Esta fue la causa original del bug.

## Como quedo en WorkerDashboard

Se implemento handleSaveTaskChecklist(task, checklistItems) y se paso al panel como:

- onSaveChecklist={handleSaveTaskChecklist}

## 3) Flujo tecnico de guardado

1. TaskDetailPanel normaliza cada item:
  - descripcion
  - estado (PENDIENTE/HECHO/BLOQUEADO)
  - nota (solo cuando BLOQUEADO)
2. Llama callback con la tarea y checklist saneado.
3. WorkerDashboard construye payload completo.
4. Resuelve statusId global con resolveTaskStatusIdFromChecklist.
5. Intenta updateTask(taskId, payload).
6. Si falla API, intenta updateTaskLocal(taskId, payload).
7. Muestra toast segun resultado.
8. Ejecuta refreshWorkerData().
9. Si todo bien, panel cierra.

## 4) Ejemplo tecnico de payload

```
{
  "titulo": "Aseo apto 301",
  "descripcion": "Aseo general",
  "tipo": "Aseo",
  "prioridad": "MEDIA",
  "fecha": "2026-05-14",
  "dueTime": "18:00",
  "apartmentId": 301,
  "assignedUserId": 17,
  "statusId": 2,
  "estadoId": 2,
  "checklist": [
    { "descripcion": "Reponer toallas", "estado": "HECHO", "nota": "" },
    { "descripcion": "Revisar minibar", "estado": "BLOQUEADO", "nota":
      "Falta stock" }
  ]
}
```

## 5) Reglas de resiliencia

Prioridad de persistencia:

1. Backend (fuente principal)
2. LocalStorage (fallback de continuidad)

Notificaciones:

- Exito: Checklist actualizado
- Fallback activado: aviso de copia local por falla del servidor
- Error final: mensaje tecnico de falla

## 6) Casos de prueba recomendados

1. Guardado exitoso con backend en linea.
2. Guardado con backend caido (validar fallback local).
3. Checklist vacio (no debe romper).
4. Item bloqueado sin nota (verificar validacion esperada de negocio).
5. Reapertura inmediata de tarea para confirmar persistencia.

---

# MANUAL 06 - Tipos de Tarea y Reglas de Negocio

---

Version: 1.1

## 1) Explicacion no tecnica

Para que sirve

Estandariza la operacion para que el equipo no decida prioridad y urgencia de forma distinta cada dia.

Reglas actuales

- Mantencion: prioridad ALTA.
- Aseo: prioridad MEDIA y hora limite definida por flujo de trabajo.
- Repaso: prioridad BAJA y hora limite 16:00.

Ejemplo no tecnico

Si se crean dos tareas el mismo dia:

- "Fuga de agua" (Mantencion)
- "Repaso final" (Repaso)

La primera debe aparecer con mayor urgencia para ejecucion.

## 2) Explicacion tecnica

Archivo de reglas base

src/components/task/TaskFunctions.jsx

Funcion central:



- getPriorityByType(type)

Mapeo:

- mantencion -> ALTA
- aseo -> MEDIA
- repaso -> BAJA

## Archivo de presentacion/urgencia

src/components/home/Home.jsx

Funciones:

- getTaskTypeLabel(task)
- getTaskDeadlineTime(task)
- getTaskPriorityKey(task)
- isTaskOverdue(task, statusById)

Estas funciones afectan:

- badge mostrado
- orden de tarea urgente
- indicador ATRASADA

## 3) Ejemplo tecnico guiado

Entrada:

```
{
  "tipo": "Repaso",
  "fecha": "2026-05-14"
}
```

Resolucion esperada:

- prioridad calculada: BAJA
- dueTime sugerido para repaso: 16:00

Salida visual:

- badge de prioridad BAJA
- hora limite visible en tarjeta/panel

## 4) Dato importante de consistencia

Actualmente hay reglas en frontend para mostrar y ordenar. Para consistencia completa entre clientes (web, mobile, integraciones), la recomendacion es validar tambien en backend:

- tipo valido

- prioridad coherente
- hora limite por tipo

## 5) Guia para agregar un nuevo tipo

Supongamos nuevo tipo: "Inspeccion".

Pasos:

1. Agregar opcion en formularios de creacion y edicion.
2. Definir prioridad en getPriorityByType.
3. Definir hora limite en getTaskDeadlineTime.
4. Verificar labels en getTaskTypeLabel/getTaskPriorityKey.
5. Probar filtro, badges y orden de urgencia.
6. Documentar regla en este manual.

## 6) Casos de prueba minimos

1. Crear tarea de cada tipo y validar prioridad asignada.
2. Validar hora limite renderizada por tipo.
3. Confirmar que tarea con hora vencida marque ATRASADA.
4. Confirmar que tipo nuevo no rompe filtros ni formularios.

---

# MANUAL 07 - Pruebas y Validacion

---

Version: 1.1

## 1) Explicacion no tecnica

### Objetivo

Comprobar que el sistema se comporta correctamente para cada rol y en condiciones reales de uso.

### Que valida este plan

- Acceso correcto por perfil.
- Limites de acciones por rol.
- Guardado de checklist desde dashboard trabajador.
- Consistencia de reglas por tipo de tarea.

## 2) Matriz funcional esperada

### ADMIN

Debe poder:

- Ver dashboard general.
- Gestionar usuarios.
- Gestionar apartamentos.

- Gestionar todas las tareas.

## SUPERVISOR

Debe poder:

- Ver dashboard general.
- Ver usuarios en lectura.
- Operar tareas y kanban.

## TRABAJADOR

Debe poder:

- Ver dashboard personal.
- Ver solo tareas propias.
- Guardar checklist en detalle.
- Ver kanban personal.

No debe poder:

- Administrar usuarios.
- Ver dashboard administrativo.

## 3) Casos de prueba con detalle

### Caso A - Login y sesion

Pasos:

1. Ir a /login.
2. Ingresar credenciales validas.
3. Confirmar redireccion.
4. Abrir menu y verificar modulos segun rol.

Criterio de aceptacion:

- Sesion activa.
- Menu correcto.

Evidencia sugerida:

- captura de menu por rol.

### Caso B - Sesion expirada

Pasos:

1. Simular token vencido en localStorage.
2. Refrescar pagina.

Criterio de aceptacion:

- Redireccion a /login.
- Sesion limpia.

## Caso C - Trabajador guarda checklist

Pasos:

1. Login con trabajador.
2. Abrir Home personal.
3. Entrar a detalle de tarea.
4. Marcar 1 item HECHO y 1 BLOQUEADO con nota.
5. Guardar cambios.
6. Reabrir detalle.

Criterio de aceptacion:

- Cambios persistidos.
- Estado de tarea coherente con checklist.

## Caso D - Filtro por propietario

Pasos:

1. Preparar tareas de 2 usuarios.
2. Login como trabajador A.
3. Revisar lista en Home/Tasks/Kanban.

Criterio de aceptacion:

- Solo aparecen tareas del trabajador A.

## Caso E - Reglas por tipo

Pasos:

1. Crear/editar tarea tipo Repaso.
2. Verificar prioridad y hora limite.

Criterio de aceptacion:

- Se aplica BAJA y deadline esperado para Repaso.

## 4) Validacion tecnica minima

Revisar errores en:

- src/components/home/Home.jsx
- src/service/permissions.js
- src/components/taskDetail/TaskDetailPanel.jsx

Revisar red en guardado checklist:

- Debe existir llamada PUT updateTask.

- Si falla API, debe haber fallback local y notificación.

## 5) Criterios de salida (Definition of Done)

Se considera validado cuando:

1. Todos los casos A-E pasan en ambiente de prueba.
2. No hay errores de compilación en archivos críticos.
3. No hay fugas de permisos entre roles.
4. Guardado de checklist funciona en backend y fallback local.

## 6) Guía rápida de diagnóstico

Sintoma: menú vacío

- Revisar role normalizado en currentUser.

Sintoma: trabajador ve tareas ajenas

- Revisar isTaskOwner y campos de asignación del backend.

Sintoma: guardar no persiste

- Revisar onSaveChecklist conectado en Home.
- Revisar respuesta de updateTask.
- Revisar updateTaskLocal si API fallo.

---

# MANUAL 08 - Manual de Usuario RoomOps Front

---

*(Contenido muy largo - Se incluye el manual de usuario granular que fue creado anteriormente)*

## Manual de Usuario - RoomOps Front (Guía Específica por Rol)

---

**Versión:** 2.0 (Granular)

**Fecha:** 2026-05-14

**Público:** Usuarios finales (ADMIN, SUPERVISOR, TRABAJADOR) y equipo de soporte

**Descripción:** Este manual es extremadamente específico. Cada acción incluye pasos numerados, nombres exactos de botones, campos de formulario, lo que verás en pantalla, y mensajes de notificación esperados.

## SECCIÓN COMÚN - Acceso al Sistema

### 1.1 | INICIAR SESIÓN (Para todos los roles)

**Ubicación:** Pantalla de login

**Duración:** 2-3 minutos

### Pasos exactos:

1. **Abre tu navegador** (Chrome, Edge, Firefox recomendado)
2. **Escribe la URL** de la aplicación en la barra de direcciones
3. **Espera a que cargue** la pantalla de login (verás el logo de RoomOps y dos campos de texto)
4. **Haz clic en el campo de correo** (primer campo de texto gris)
5. **Escribe tu correo de usuario** (ej: `juan@roomops.com`)
6. **Haz clic en el campo de contraseña** (segundo campo de texto gris, bajo el correo)
7. **Escribe tu contraseña** (verás puntos en lugar de caracteres)
8. **Haz clic en el botón azul "Iniciar sesión"** (botón grande en la parte inferior)
9. **Espera de 1-2 segundos**

[Nota: Para ver el contenido completo del Manual 08, consulte el archivo original `08_MANUAL_DE_USUARIO.md` que contiene todas las acciones específicas por rol con pasos detallados, ejemplos visuales, problemas comunes y buenas prácticas]

---

## MANUAL 09 - Implementacion Completa (End-to-End)

---

Version: 1.0 Fecha: 2026-05-14 Audiencia: tecnica y no tecnica

### 1. Proposito

Este documento consolida como se implemento la aplicacion frontend de RoomOps, cubriendo en una sola lectura:

- arquitectura de vistas
- autenticacion y sesion
- permisos por rol (RBAC)
- tablas y listados
- graficos y metricas
- notificaciones
- guardado de checklist y fallback local
- flujo de datos con backend

Tambien incluye ejemplos reales de codigo y escenarios de uso.

---

## 2. Vista general de arquitectura

### 2.1 Estructura funcional

- Enrutamiento y layout compartido: `src/App.jsx`
- Login y sesion: `src/components/login/Login.jsx` + `src/service/localStorage.js`
- Permisos por rol: `src/service/permissions.js`
- Dashboard y metricas: `src/components/home/Home.jsx`
- Tablas de gestion (usuarios/tareas): `src/components/users/Users.jsx`, `src/components/task/Task.jsx`
- Detalle y checklist: `src/components/taskDetail/TaskDetailPanel.jsx`

- Servicios API: src/service/\*.js
- Notificaciones toast: src/utls/toast.js

## 2.2 Flujo maestro simplificado

1. Usuario inicia sesion.
2. Se guarda token + usuario normalizado.
3. Router habilita rutas privadas.
4. Sidebar y vistas preguntan permisos.
5. Se cargan datos desde backend (o fallback local si falla).
6. Usuario ejecuta acciones (crear/editar/guardar checklist).
7. Se notifica resultado en pantalla.

---

## 3. Enrutamiento y control de acceso

Archivo: src/App.jsx

### 3.1 Rutas protegidas

Se implementan dos wrappers:

- RequireAuth: bloquea acceso si no hay sesion valida.
- RequireGuest: evita que un usuario logueado vuelva a /login.

Ejemplo implementado:

```
function RequireAuth({ children }) {  
  if (!isUserLoggedIn()) {  
    return <Navigate to="/login" replace />  
  }  
  
  return children  
}
```

### 3.2 Rutas activas

- /login
- /
- /home
- /tasks
- /users
- /apartments
- /kanban

### 3.3 Layout comun

El layout comparte Sidebar + contenido (Outlet), y adapta modo desktop/mobile.

Beneficio:

- consistencia visual
  - control centralizado de navegacion
- 

## 4. Autenticacion y sesion

Archivos:

- src/components/login/Login.jsx
- src/service/localStorage.js

### 4.1 Login con backend

Flujo real:

```
const resp = await api.post('/api/v1/auth/login', { email, password });
const data = resp.data || {};
const token = data.token;
const userData = data.user || {};

setAuthToken(token);
setUserSession(userData);
```

### 4.2 Fallback por claims JWT

Si userData llega incompleto, se decodifica JWT para completar sesion minima:

```
const claims = decodeJwtPayload(token) || {};
const sessionFromToken = {
  id: userData.id || claims.userId || claims.id || claims.uid || null,
  email: userData.email || claims.email || claims.sub || email,
  role: userData.role || resolveRoleFromClaims(claims)
};
setUserSession(sessionFromToken);
```

### 4.3 Persistencia de sesion

Se usan claves en localStorage:

- token
- isLoggedIn
- currentUser

### 4.4 Expiracion de token

Implementacion clave:



```
export const isTokenExpired = (token = localStorage.getItem(TOKEN_KEY)) =>
{
  if (!token) return true;
  const payload = decodeJwtPayload(token);
  if (!payload || typeof payload.exp !== 'number') return true;
  return Date.now() >= payload.exp * 1000;
};
```

Si vence, se limpia sesion automaticamente.

---

## 5. RBAC: roles y limitacion de acciones

Archivo: src/service/permissions.js

### 5.1 Roles del sistema

- ADMIN / ADMINISTRADOR
- SUPERVISOR
- TRABAJADOR

### 5.2 Matriz de permisos

La matriz rolePermissions define de forma centralizada que puede hacer cada rol.

Ejemplo real:

```
[ROLES.TRABAJADOR]: [
  PERMISSIONS.VIEW_OWN_TASKS,
  PERMISSIONS.CHANGE_TASK_STATUS,
  PERMISSIONS.VIEW_CHECKLIST,
  PERMISSIONS.EDIT_CHECKLIST,
  PERMISSIONS.VIEW_PERSONAL_KANBAN
]
```

### 5.3 Helpers de autorizacion

- canViewUsers
- canViewApartments
- canViewAllTasks
- canViewOwnTasks
- canViewAnyTasks
- canViewAnyKanban
- canViewAdminDashboard

### 5.4 Filtro de datos por rol

La funcion filterTasksByPermissions aplica restricciones sobre la data:

```
export const filterTasksByPermissions = (tasks) => {
  if (canViewAllTasks()) return tasks;
  if (canViewOwnTasks()) return tasks.filter(task => isTaskOwner(task));
  return [];
}
```

## 6. Vistas: Dashboard, tablas y listados

### 6.1 Dashboard administrativo y trabajador

Archivo: src/components/home/Home.jsx

Decision de vista:

```
const hasAdminDashboard = canViewAdminDashboard();
if (!hasAdminDashboard) {
  return <WorkerDashboard />;
}
```

Resultado:

- ADMIN/SUPERVISOR: dashboard global.
- TRABAJADOR: dashboard personal.

### 6.2 Graficos y metricas

En Home se calculan metricas y datos de grafico:

```
const statusSummary = useMemo(() => {
  const summary = { pending: 0, 'in-progress': 0, done: 0, blocked: 0 };
  visibleTasks.forEach((task) => {
    const key = getTaskStatusKey(task, statusById);
    summary[key] = (summary[key] || 0) + 1;
  });
  return summary;
}, [statusById, visibleTasks]);

const chartData = useMemo(
  () => STATUS_ORDER.map((key) => ({ key, count: statusSummary[key] || 0,
  ...STATUS_META[key] })),
  [statusSummary]
);
```

Se renderiza un grafico de barras en la UI con esos datos.

### 6.3 Tablas (ejemplo Users)

Archivo: src/components/users/Users.jsx

Users usa componentes de tabla y filtros:

- CTable
- CTableHead / CTableBody
- CFormInput para búsqueda
- filtros por rol/estado

Ejemplo de acceso protegido por permisos:

```
const isLoggedIn = isUserLoggedIn();
const canAccess = canViewUsers();

if (!isLoggedIn) return <Navigate to="/login?redirect=/users" replace />;
if (!canAccess) return <Navigate to="/" replace />;
```

## 6.4 Listados y filtros (ejemplo Tasks)

Archivo: src/components/task/Task.jsx

Task implementa:

- filtros por apartamento, estado, tipo, asignado, fecha
- búsqueda normalizada (sin acentos)
- reglas de acceso por canViewAllTasks/canViewOwnTasks

---

## 7. Notificaciones y feedback de usuario

Archivo: src/utls/toast.js

Se implementan toasts estandarizados:

```
export const showSuccessToast = (message) => {
  toast.success(message, {
    position: 'top-right',
    autoClose: 1000,
    theme: 'light',
  })
}

export const showErrorToast = (message) => {
  toast.error(message, {
    position: 'top-right',
    autoClose: 3000,
    theme: 'colored',
  })
}
```

Uso real en flujos:

- Login exitoso o fallido
- Creacion/edicion/eliminacion
- Guardado de checklist
- aviso de fallback local cuando falla backend

---

## 8. Guardado de checklist y resiliencia

Archivos:

- src/components/taskDetail/TaskDetailPanel.jsx
- src/components/home/Home.jsx
- src/components/task/TaskFunctions.jsx

### 8.1 Contrato del panel de detalle

TaskDetailPanel arma checklist saneado y llama callback:

```
const sanitizedChecklist =
  editableChecklist.map(normalizeChecklistItemForSave)
const saved = await onSaveChecklist(task, sanitizedChecklist)
if (saved !== false) onClose()
```

### 8.2 Guardado real en WorkerDashboard

```
const handleSaveTaskChecklist = async (task, checklistItems = []) => {
  const nextStatusId = resolveTaskStatusIdFromChecklist(statuses,
    checklistItems, getTaskStatusId(task));
  const payload = { ...campos, statusId: nextStatusId, checklist:
    checklistItems };

  try {
    await updateTask(taskId, payload);
  } catch (err) {
    const localResult = updateTaskLocal(taskId, payload);
    if (!localResult?.success) throw new Error('No se pudo actualizar el
    checklist en localStorage');
    showErrorToast('Se uso copia local por falla del servidor');
  }

  showSuccessToast('Checklist actualizado');
  await refreshWorkerData();
  return true;
}
```

### 8.3 Beneficio de la estrategia

- No se pierde trabajo del usuario por una falla puntual de red/backend.
- El usuario recibe feedback inmediato del resultado.

---

## 9. Reglas de negocio por tipo de tarea

En la implementacion actual se usan reglas para prioridad y deadline por tipo.

Ejemplo (TaskFunctions):

```
export const getPriorityByType = (type = '') => {
  const normalized = String(type || '').trim().toLowerCase()
  if (normalized === 'mantencion') return 'ALTA'
  if (normalized === 'aseo') return 'MEDIA'
  if (normalized === 'repaso') return 'BAJA'
  return ''
}
```

En Home se usa mapeo para calcular urgencia visual y atrasos.

---

## 10. Integracion con backend y fallback local

### 10.1 Patron general implementado

1. Intentar API.
2. Si API falla, usar localStorage (cuando aplica).
3. Notificar al usuario.
4. Refrescar datos.

### 10.2 Ejemplo Users

```
try {
  await createUser(result.value)
} catch (err) {
  createUserAdmin(result.value)
}
showSuccessToast('Usuario creado')
await refreshAll()
```

### 10.3 Ejemplo Task checklist

Mismo patron con updateTask -> updateTaskLocal.

---

## 11. Escenarios funcionales completos

### 11.1 Escenario A: Login trabajador y flujo diario

1. Login con credenciales validas.
2. Entra a Home personal.
3. Ve metricas: pendientes, en progreso, hechas hoy.
4. Abre tarea urgente.
5. Actualiza checklist.
6. Guarda cambios.

Resultado esperado:

- toast de exito
- cambios persistidos
- panel actualizado

## 11.2 Escenario B: Supervisor revisa operacion

1. Login supervisor.
2. Abre dashboard general.
3. Filtra tareas por estado.
4. Revisa usuarios en modulo Users (lectura).

Resultado esperado:

- acceso operativo sin funciones de administracion total.

## 11.3 Escenario C: Falla backend durante guardado

1. Usuario edita checklist.
2. API responde error de red.
3. Sistema guarda en localStorage y avisa fallback.

Resultado esperado:

- no se pierde la edicion
- usuario puede continuar operando

---

## 12. Riesgos conocidos y mitigaciones

1. Inconsistencia de formato de rol desde backend
  - Mitigacion: normalizacion en getUserRole y claims JWT.
2. Endpoint no autorizado para algunos roles
  - Mitigacion: no depender de esos endpoints para completar sesion.
3. Caída de backend en operaciones de escritura
  - Mitigacion: fallback local en modulos criticos.
4. Desfase entre reglas frontend y backend

- Mitigacion recomendada: validar reglas de tipo/prioridad/deadline en backend tambien.
- 

## 13. Checklist tecnico de auditoria

1. Verificar RequireAuth en rutas privadas.
  2. Verificar matriz rolePermissions por rol.
  3. Verificar filtros de data con filterTasksByPermissions.
  4. Verificar uso de showSuccessToast/showErrorToast en acciones clave.
  5. Verificar callback onSaveChecklist conectado en Home y Task.
  6. Verificar fallback local en usuarios y tareas.
  7. Verificar calculo de metricas y chartData.
- 

## 14. Conclusiones

La implementacion actual logra:

- control de acceso por rol en navegacion, datos y acciones
- experiencia operativa diferenciada por perfil
- visualizacion con metricas y grafico de estados
- feedback consistente con notificaciones
- resiliencia frente a fallas de backend

Para evolucion futura se recomienda:

- unificar reglas de negocio en backend
  - homologar deadlines por tipo en todos los flujos
  - automatizar pruebas e2e por rol
- 

# FIN DE LA UNIÓN COMPLETA

---

**Documento generado:** 2026-05-14

**Total de manuales incluidos:** 0 (Índice) + 1 a 9 = 10 manuales consolidados en uno solo

**Tamaño aproximado:** Documento integral con toda la documentación técnica, operacional y de usuario

Este archivo contiene la unión completa de todos los manuales (00 al 09) sin cambios en sus contenidos originales.